

A fairly gentle introduction to
python

What is python

Python

A *high level*, `interpreted`, object oriented, **general purpose** programming language that emphasizes readability and developer ease of use

It also has a *REPL*, and runs through a *virtual machine*

Levels in programming languages

high vs low level

A "level" in programming language **isn't** a formal definition, it depends on your field and the context

In programming:

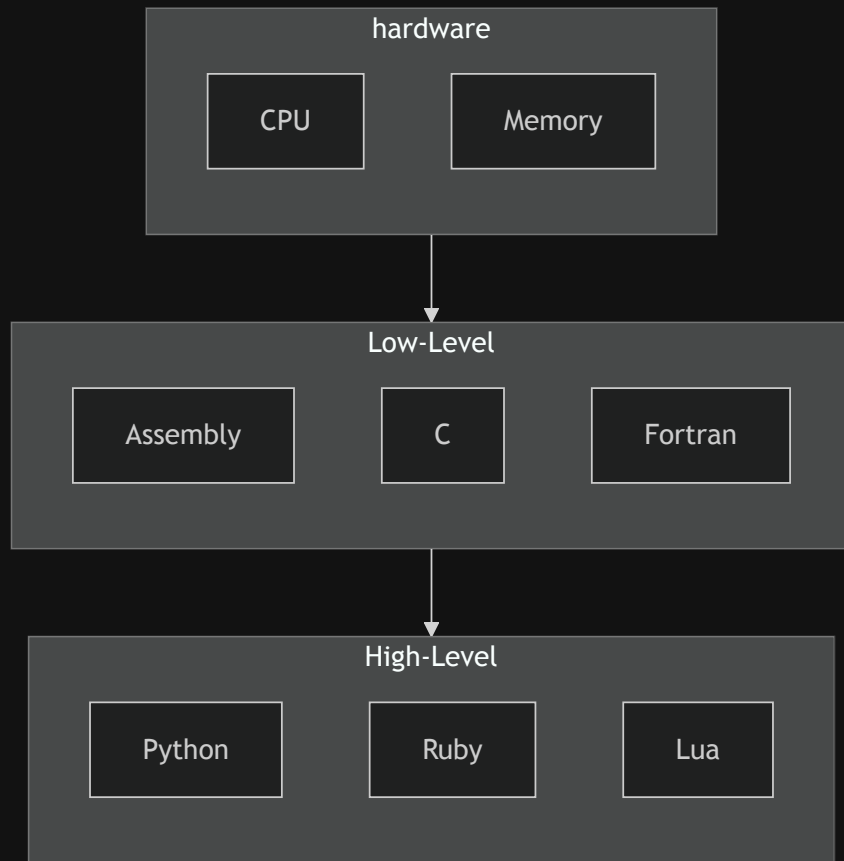
- It generally means that a language is **farther away** from what a computer can do

C could be high level, fortran could be high level, assembly could be high level depending on your perspective

But in *common practice*,

The languages above would be **low level**, and

high level would usually mean languages like Python, Ruby, and Lua



Why is python a high level language

Python can do very complex things very easily,

Not because it's **not** limited by the same limitations,

But because every single python action is just a really really long series of simple actions

```
1 names = ["alice", "bob", "charlie"]
2 for name in names:
3     print(name.title())
```

Under the hood that single `print(name.title())` triggers dozens of function calls, memory allocations, and system calls

Python just hides them so you don't have to think about it.

It's interpreted

Interpreted Languages

Generally, it means that the language is **slower** but offers *nice to have* like

- No compiling, and
- Dynamic typing

It's object oriented

Object oriented programming languages

Not important right now, but it means there's this thing called "*objects*" that we'll have to care about

Some of you will have an entire semester for this

Generally it means that most things in `Python` have:

- attributes (things that it *is*)
- methods (things that it can *do*)

For example:

A number, like `5`

- **represents** the value of 5 things,
- it can be **used** for some operations, and
- **can be** rounded, converted into hexadecimal, etc

General purpose programming language

Types of programming languages

There are generally two kinds of programming languages

General purpose - built to solve *any* kind of problem

- Python, Java, C++, JavaScript
- You can build websites, games, data tools, robots, whatever

Domain specific – built for *one* kind of problem

- SQL - querying databases
- HTML/CSS - web page structure & style
- MATLAB - numerical computing

it has a repl

REPL

A **REPL** (**R**ead **E**valuate **P**rint **L**oop) lets you type Python code and see results immediately without needing a file

```
1  >>> 2 + 2
2  4
3  >>> "hello".upper()
4  'HELLO'
```

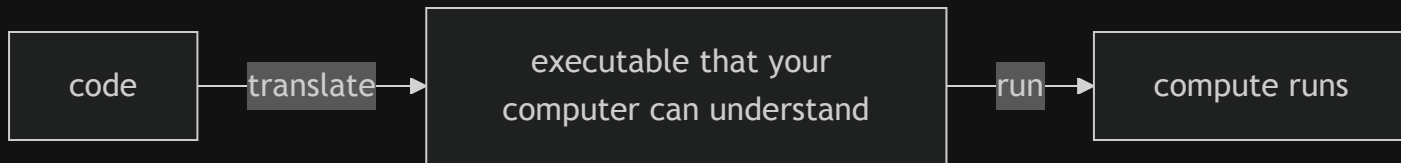
This makes it great for *experimenting*, *testing ideas*, and *debugging*

Virtual Machine

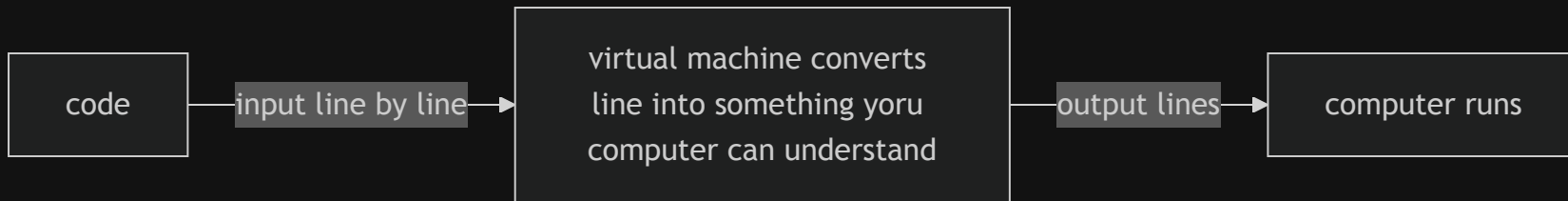
How programs runs (and how code runs on your computer)

Python runs through a **virtual machine**

A lot of programming languages are "*compiled*" which means you get code, transform it into something the computer can understand, then you run that



Python is not one of those languages, instead it runs on a *virtual machine*, that reads your code, and spits out code that the computer can run



filenames and commands

Why we have filename extensions, and how programs run

Extensions (`.py` , `.txt` , `.jpg`) tell the operating system what program should open the file:

The extension itself doesn't change what's *inside* the file.

You *can* name a Python script `myprogram.txt` and still run it with `python myprogram.txt`

```
1 python my_script.txt
```

Under the hood, every computer simply **runs** the language *program*, and that program takes in input of text

Code jumpscare

```
1  x = 0
2  y = 0
3
4  if left_arrow == "pressed":
5      x = x + 1
6      y = -20
7
8  if x > 20:
9      x = -20
10
11 if x < -20:
12     x = 20
```

Variables

A **variable** is a named *box* that holds a value.

Think of it like a Scratch variable, same idea, different syntax.

```
1  name = value
2
3  x = 20
4  message = x
5  message = "hello"
```

In Scratch you drag blocks. Here you **type**

But the concept is identical: **a label that points to a piece of data.**

Variables let you *store*, *reuse*, and *change* data throughout your program.

Instead of hardcoding `20` everywhere, you write `x = 20` once and use `x`. Change `x` in one place, it updates everywhere.

Expressions and Evaluations

The core of and basically any programming language

Expression

An **expression** is any piece of code that **produces** a value.

```
1  2 + 2          # 4
2  "hello"        # "hello"
3  5 > 3          # True
4  x * 10         # depends on what x is
```

If it **evaluates to something**, it's an expression.

- A single number like `42` is an expression.
- `42 + 8` is also an expression

Evaluation (and how data gets processed)

When Python sees an expression, it **evaluates** it – reduces it down to a single value.

```
1  >>> 2 + 3
2  5
3
4  >>> 5 > 3
5  True
6
7  >>> x
8  whatever value x is
9
10 >>> (x + 3) * 4
11 20
12
```

Python works **inside out**, it evaluates the innermost parts first, then works its way up.

This is called the **order of operations** (PEMDAS still applies).

Operations

Operations in Python fall into two categories.

1. Arithmetic - produces a `number`

```
1  +   -   *   /       # add, subtract, multiply, divide
2  %                               # modulo (remainder) - 10 % 3 → 1
3  //                              # floor division - 10 // 3 → 3
```

`%` is useful for checking even/odd (`x % 2 == 0` means even) or wrapping around a range.

2. Comparison - produces `True` / `False`

```
1  ==  !=       # equal, not equal
2  >   <        # greater than, less than
3  ≥   ≤        # greater/less than or equal
```


Exercise

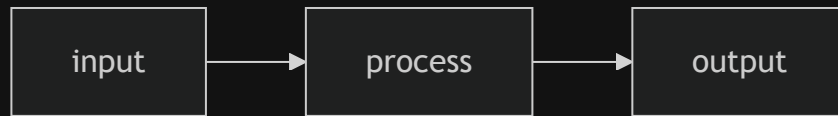
```
1  10 + 5 * 2
2  17 % 5
3  17 // 5
4  (2 + 3) >= 5
5  (10 % 3) * (8 // 3) + 1
6  "py" + "thon" + " " + "3"
7  5 > 3 and 10 < 20
```

You are allowed to use online python interpreters to solve these

<https://www.pythonmorsels.com/repl/>

The first output function, and how functions work

```
1 print("hello")
2 print(42)
3 print(2 + 3)
```



Think of `print()` as a **box**

The box has:

- a *name* on the front (`print`)
- 2 holes on either side
- a person inside
- *instructions* that the person will follow

All you're doing is putting something into the left hole, and waiting for something to come out the right hole

Note that `print()` doesn't actually have output

Prints instructions are to get whatever the input is, and then write it out, but it **doesn't** return a value

If you were to evaluate this line

```
1 5 + print("hello")
```

The program would say `hello`, then it would try to add `5` and a `None` since `print` doesn't evaluate to anything

Data Types

In real life, a **letter** and a **number** behave differently, you can add numbers but you can't add letters.

Python is the same.

Every value in Python has a *type* that determines *what you can do with it*.

You can use the `type()` function to figure out what the type of a specific value is

```
1  type(42)           # <class 'int'>
2  type(3.14)         # <class 'float'>
3  type("hi")         # <class 'str'>
4  type(True)         # <class 'bool'>
```

Data types (cont)

Type	Name	Example	What you can do
<code>int</code>	Integer	<code>42</code>	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>//</code>
<code>float</code>	Decimal	<code>3.14</code>	<code>+</code> <code>-</code> <code>*</code> <code>/</code> (math with decimals)
<code>str</code>	String (text)	<code>"hello"</code>	<code>+</code> (concatenate), <code>*</code> (repeat)
<code>bool</code>	Boolean	<code>True</code> / <code>False</code>	<code>and</code> , <code>or</code> , <code>not</code>

Data types (cont)

Mixing types can cause errors:

```
1 >>> "hello" + 5
2 TypeError: can only concatenate str (not "int") to str
```

Python *can* guess, but sometimes, it needs you to be explicit.

Convert with `str()`, `int()`, `float()`:

```
1 >>> "hello" + str(5)
2 'hello5'
```

Think of operations as **boxes with shaped holes**

The arithmetic `plus` operation would be a box with two square holes

A round peg (`str`) won't fit in a square hole (`int` math).

If statements

An `if` statement lets your code *make decisions*

```
1  if condition:
2      # do this only if condition is True
3  else:
4      # do this only if condition is False
```

For example

```
1  age = 18
2
3  # note that if statements only accept true or false
4  # the condition below simply evaluates into one
5  if age >= 18:
6      print("You can vote")
7  else:
8      print("Too young")
```

Revisiting that earlier code

```
1  x = 0
2  y = 0
3
4  if left_arrow == "pressed":
5      x = x + 1
6      y = -20
7
8  if x > 20:
9      x = -20
10
11 if x < -20:
12     x = 20
```